

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Построение и анализ алгоритмов»
Тема: «Поиск подстроки в строке»

Студент гр. 4345

Андросов М.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритм Кнута-Морриса-Пратта.

Задание

Реализуйте алгоритм КМП и с его помощью для заданных шаблона P ($|P| \leq 25000$) и текста T ($|T| \leq 5000000$) найдите все вхождения P в T .

Вход:

- Первая строка — P
- Вторая строка — T

Выход:

индексы начал вхождений P в T , разделённые запятой; если P не входит в T , то вывести -1.

Выполнение работы

Разработан алгоритм Кнута-Морриса-Пратта.

Описание алгоритма

1) Для строки p строится массив arr длины n , где i -ый элемент это длина наибольшего префикса, который одновременно является суффиксом.

2) С использованием массива, полученного на предыдущем шаге алгоритма выполняется поиска строки P в строке T :

Начинаем с первого элемента в T , i — индекс строки j — индекс подстроки.

а) Сравниваем $T[i]$ и $P[j]$

б) если элементы совпадают, оба индекса увеличиваются ($i++$, $j++$). При несовпадении j откатывается к значению $arr[j-1]$.

в) если $j == m$ (длине P) фиксируем вхождение в индексе $i - m + 1$. Присваиваем $j = arr[j-1]$ (для следующего вхождения).

Оценка сложности алгоритма

Временная сложность - $O(n+m)$

Пространственная - $O(n + m)$

Разработанный программный код см. в приложении А.

Тестирование

Таблица 1 — Результаты тестирования.

№	Входные данные	Выходные данные
1	ab abab	0,2
2	abcd aaaa	-1
3	abc abc	0
4	ab ababcab	0,2,5

Вывод

В ходе работы был реализован и протестирован рекурсивный алгоритм Кнута-Морриса-Пратта.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.py

```
def prefix_func(p):
    n = len(p)
    arr = [0] * n
    for i in range(1,n):
        j = arr[i-1]
        while j > 0 and p[i] !=p[j]:
            j = arr[j-1]
        if p[i] == p[j]: j +=1
        arr[i] = j

    return arr


def kmp_search(p, t):
    arr = prefix_func(p)
    n = len(t)
    m = len(p)
    ans = []

    j = 0
    for i in range(n):
        while j > 0 and t[i] != p[j]:
            j = arr[j-1]

        if t[i] == p[j]:
            j +=1

        if j == m:
```

```
    ans.append(i - m + 1)
    j = arr[j-1]
```

```
return ans
```

```
if __name__ == "__main__":
    p = input().strip()
    t = input().strip()
    ans = kmp_search(p,t)
    if len(ans) == 0: print(-1)
    else: print(",".join(map(str,ans)))
```